



Solar ASIC
AUTOMATISATION

Anleitung Installations- Komplettanleitung

Automatisieren Sie Ihre ASIC-Miner mit Solarenergie.
Kein Netzbezug tagsüber | Solar-Mining | Bemarkt-sicher.

Solar

Krypto

Home Assistant

Automation V1.0.37



VERSION

1.0.37

AUTOR

Halo44

LIZENZ

Frei & Offen

HARDWARE

Refoss + HA

Inhalt

- | | |
|-----------|--|
| 01 | Das Konzept
Warum Solar ASIC? Bearmarkt + Solar |
| 02 | Architektur
Überblick über das Gesamtsystem |
| 03 | Hardware & Verkabelung
Refoss EM06P, Steckdosen, CT-Platzierung |
| 04 | Home Assistant
Integration und Grundkonfiguration |
| 05 | configuration.yaml
Sensoren, Templates, prioritäre ASICs, Erträge |
| 06 | Automation V1.0.37
Multi-Priorität, Anti-Jojo, kontinuierliches SHED |
| 07 | Tagesbericht
Zuverlässige 20-Uhr-Meldung (history_stats) |
| 08 | HTML-Dashboard
Echtzeit-Oberfläche, kWh-Diagramme, i18n |
| 09 | Mining-Pools
7 Pools: F2Pool, K1Pool, 2Miners, ViaBTC... |
| 10 | Fernzugriff
Tailscale VPN - Zugang über 4G / WiFi |
| 11 | Fehlerbehebung
FAQ und Problemlösung |

01 Das Konzept

Warum Solar ASIC?

Im Krypto-Bearmarkt ist Mining mit Netzstrom zu 0,20 EUR/kWh oft ein Verlustgeschäft. Die Lösung: nur den Solar-Uberschuss Ihrer Anlage nutzen. Solange es Solar-Uberschuss gibt, laufen die ASICs kostenlos. Sobald der Uberschuss sinkt, schalten sie sich automatisch ab, um nie Netzstrom zu beziehen.

Der automatische Tageszyklus

Zeit	Verhalten
Morgen	Sonnenaufgang -> wachsender Uberschuss. Kleine ASICs starten nacheinander (5s-Kaskade).
Tag	Maximaler Uberschuss. Die Automation wechselt zu den grossen prioritaren ASICs (S19, S21...). 100% Solar-Mining, kein Netz.
Abend	Uberschuss sinkt. ASICs schalten sich schrittweise ab (umgekehrte Reihenfolge, LIFO).
Nacht	Alle ASICs aus. Kein Netzbezug.

Hauptunterschied zu v1: die V1.0.37-Automation verwaltet eine **Liste prioritärer ASICs**. Morgens fullen kleine ASICs den Uberschuss; sobald genug Sonne fur einen grossen Miner da ist, werden die kleinen kurz abgeschaltet, wahrend er startet, und kehren dann im verbleibenden Uberschuss zuruck. Automatische Kaskade fur 1, 2, 3 grosse ASICs oder mehr.

Drei Saulen

- **Solar zuerst:** die PV-Produktion wird in Echtzeit uber 6 CT-Zangen gemessen.
- **Intelligente Entscheidung:** Home Assistant berechnet den Uberschuss und steuert jede Steckdose.
- **Anti-Jojo:** Verzogerungen und Cooldowns verhindern Ein-/Ausschalten durch ziehende Wolken.

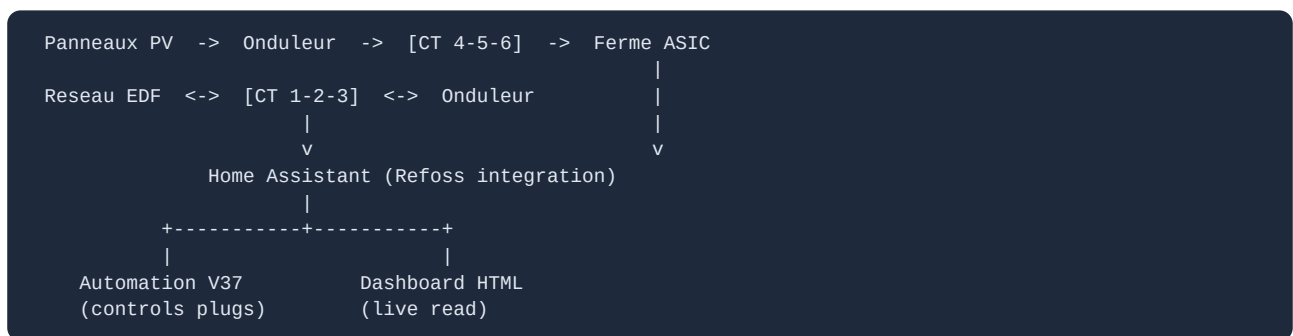
02 Architektur

Systemüberblick

Fünf Glieder verbinden Ihre Module mit Ihren ASICs. Jedes hat eine genaue Rolle in der Mess- und Entscheidungskette.

Glieder	Rolle
1. PV-Module	Erzeugen die Energie (z.B. 12 kWp)
2. Refoss EM06P	6 CT-Zangen messen Netz (CT 1-2-3) und ASIC-Farm (CT 4-5-6)
3. Home Assistant	Gehirn: berechnet den Überschuss, entscheidet welche ASICs laufen (Raspberry Pi)
4. Steckdosen	Führen aus: WiFi / Zigbee / Matter (Tuya, Shelly...)
5. ASIC-Miner	Minen: IceRiver, Goldshell, Antminer...

Datenfluss



Vorteil	Detail
Automatisch 24/7	Die Automation läuft durchgehend, Zyklus alle 5 Sekunden
Anti-Jojo	30s Cooldown + 10 min Prioritäts-Boot
Fernzugriff	Tailscale VPN zur Kontrolle über 4G
Netzkosten	0 EUR für Mining tagsüber

03

Hardware & Verkabelung

Verkabelungsschema - Refoss CT-Zangen

Der Refoss EM06P hat 6 Messkanäle (6 Stromzangen). Ihre Platzierung ist entscheidend, damit die Überschussberechnung stimmt.

Zangen	Platzierung	Messung
CT 1, 2, 3	Netzkabel, zwischen Zähler und Wechselrichter	Netzbezug/-einspeisung (Vorzeichen: + Bezug, - Einspeisung)
CT 4, 5, 6	Ausgangskabel Wechselrichter (AC-Seite)	Verbrauch der ASIC-Farm (immer positiv)

WICHTIG: das Spannungskabel des Refoss (L1 / L2 / L3 / N) muss am selben Stromkreis wie CT 1-2-3 angeschlossen sein, sonst sind die Leistungsmessungen falsch.

Hardware-Liste

Element	Empfohlenes Modell	Verwendung
6-Kanal-CT-Zähler	Refoss EM06P (oder 2x Shelly 3EM)	Solar + Netz messen
Steckdosen	Tuya / Shelly / Matter, 1 pro ASIC	Gesteuertes Ein/Aus
Mini-PC / Pi	Raspberry Pi 3B+ minimum oder PC	Hostet Home Assistant
ASIC-Miner	IceRiver, Goldshell, Antminer...	Nach Budget

Steckdosen-Tipp: prüfen Sie die max. Stromstärke jeder Steckdose. Ein Antminer S19 (~3400W) braucht eine dedizierte 16A-Steckdose. Kleine IceRiver (~100W) laufen an 10A-Standard.

04 Home Assistant

Grundkonfiguration

Bevor Sie die Sensoren programmieren, integrieren Sie die Hardware und bereiten die HA-Umgebung vor.

Vorbereitende Schritte

- 1. Refoss-Integration:** Einstellungen -> Geräte & Dienste -> Hinzufügen -> Refoss. Die 6 Sensoren `sensor.em06_channel_X_power` erscheinen.
- 2. Steckdosen-Integration:** fügen Sie Ihre Steckdosen hinzu (Tuya, Shelly...). Notieren Sie deren `entity_id` (Entwicklertools -> Zustände -> Filter 'switch.').
- 3. Langzeit-Token:** Profil -> Sicherheit -> Token erstellen. Für das Dashboard kopieren.
- 4. www-Ordner:** erstellen Sie `/config/www/` falls nicht vorhanden (für das Dashboard).

CORS: damit das HTML-Dashboard auf die HA-API zugreift, fügen Sie den `http`-Block mit `cors_allowed_origins` in `configuration.yaml` hinzu (siehe Abschnitt 05).

Recorder (Verlauf)

Das Dashboard zeigt Tages-/Wochen-/Monatsdiagramme. Dafür muss HA den Sensorverlauf mindestens 31 Tage aufbewahren:

```
recorder:
  purge_keep_days: 31
  commit_interval: 60
  include:
    entities:
      - sensor.production_solaire
      - sensor.consomption_ferme
      - sensor.consomption_reseau
      - sensor.injection_reseau
      - sensor.asic_allumes_count
```

05 configuration.yaml

Berechnete Sensoren (Jinja2-Templates)

Diese Sensoren wandeln die CT-Rohmessungen in nutzbare Werte um: Solarproduktion, Überschuss, Netzbezug/-einspeisung, ASIC-Zahlung.

Was Sie anpassen müssen

- **sensor.em06_channel_X_power** -> die echten Namen Ihrer Refoss-Sensoren
- **switch.asic_small_1 ...** -> die entity_id Ihrer Steckdosen
- **100, 360, 3400** -> der reale Verbrauch (W) jedes ASIC

```
template:
- sensor:
  - name: "reseau_total"          # CT 1-2-3 (+ import / - export)
    unit_of_measurement: "W"
    state: >
      {% set p1 = states('sensor.em06_channel_1_power')|float(0) %}
      {% set p2 = states('sensor.em06_channel_2_power')|float(0) %}
      {% set p3 = states('sensor.em06_channel_3_power')|float(0) %}
      {{ (p1 + p2 + p3) | round(1) }}

  - name: "injection_reseau"      # surplus to grid
    state: >
      {% set p = states('sensor.reseau_total')|float(0) %}
      {{ (p * -1) if p < 0 else 0 }}

  - name: "consommation_ferme"    # CT 4-5-6 (ASIC load)
    state: >
      {% set p4 = states('sensor.em06_channel_4_power')|float(0) %}
      {% set p5 = states('sensor.em06_channel_5_power')|float(0) %}
      {% set p6 = states('sensor.em06_channel_6_power')|float(0) %}
      {{ [p4 + p5 + p6, 0] | max | round(1) }}

  - name: "production_solaire"    # farm + export
    state: >
      {% set f = states('sensor.consommation_ferme')|float(0) %}
      {% set i = states('sensor.injection_reseau')|float(0) %}
      {{ (f + i) | round(1) }}
```

Prioritare ASICs (Quelle der Wahrheit)

Das Herz des V1.0.37-Systems: eine einzige Liste definiert, welche ASICs prioritar sind (die grossen Miner). Die Automation liest diese Liste und regelt alles automatisch. Um einen ASIC hinzuzufügen (z.B. einen neuen S21), fügen Sie hier einfach eine Zeile hinzu - keine Änderung der Automation.

```
- name: "asic_prio_config"
  state: "ok"
  attributes:
    asics: >-
      {{ [
        {'entity': 'switch.asic_big_1', 'power': 3400, 'name': 'S19'}
      ] }}
  # Multiple priorities:
  # {'entity': 'switch.s19', 'power': 3450, 'name': 'S19'},
  # {'entity': 'switch.s21_1', 'power': 3500, 'name': 'S21 #1'},
  # {'entity': 'switch.s21_2', 'power': 3500, 'name': 'S21 #2'}
```

Fünf abgeleitete Sensoren (so kopieren) berechnen automatisch: die für den nächsten Start benötigte Leistung, den nächsten einzuschaltenden ASIC, den zuletzt eingeschalteten und die Anzahl aktiver prioritärer ASICs.

Sensor	Rolle
<code>asic_prio_power_needed</code>	Gesamtleistung für den Start der nächsten Priorität
<code>asic_prio_next_entity</code>	entity_id des nächsten einzuschaltenden ASIC (FIFO)
<code>asic_prio_last_entity</code>	entity_id des zuletzt eingeschalteten (zuerst aus, LIFO)
<code>asic_prio_count_on</code>	Anzahl aktuell eingeschalteter prioritärer ASICs
<code>binary_sensor.asic_prio_add_ready</code>	AN wenn die Solarproduktion die nächste Priorität deckt

Mining-Erträge in Euro

Zwei Template-Sensoren multiplizieren die Produktion jedes Pools (in Krypto) mit dem über CoinGecko abgerufenen Euro-Preis, um die realen Erträge in EUR anzuzeigen.

```
- platform: rest          # crypto prices, 1 call / 10 min
  name: "crypto_prices_eur"
  resource: "https://api.coingecko.com/api/v3/simple/price
    ?ids=bitcoin,kaspa,alephium&vs_currencies=eur"
  scan_interval: 600
  json_attributes: [bitcoin, kaspa, alephium]
```

06

Automation V1.0.37

Multi-Prioritat-Logik

Die V37-Automation läuft im Single-Modus (eine Instanz gleichzeitig) mit 5 Auslosern. Sie enthält keinen fest codierten ASIC-Namen: alles kommt aus sensor.asic_prio_config.

Ausloser	Bedingung	Aktion
cycle_normal	Alle 5s	Kontinuierliche Regelung: fugt kleine ASICs hinzu/entfernt sie
maybe_prio_add	Prod >= Bedarf, 1 min	Startet den nächsten prioritären ASIC
sustained_import	Netz > 250W, 5 min	Reduziert kleine ASICs
production_drop	Prod < 2800W, 5 min	Schaltet die letzte Priorität ab (Abend)
endofday_cleanup	Netz > 50W, 10 min	Schaltet alle verbleibenden kleinen ASICs ab (Nacht)

Start-Sequenz einer Priorität

Wenn die Solarproduktion die Leistung des nächsten grossen ASIC für 1 Minute deckt:

- Alle kleinen ASICs werden abgeschaltet (230V-Bus freigeben)
- Nach 25s startet der grosse ASIC
- Für 10 min ist das Hinzufügen kleiner ASICs **blockiert** (der grosse fährt ~5 min hoch)
- Nach Stabilisierung kehren die kleinen ASICs im verbleibenden Überschuss zurück

Warum die 10-min-Blockade? Ein grosser ASIC (S19) zieht seine 3400W nicht sofort: er braucht ~5 min zum Hochfahren. Ohne Blockade würde der ADD-Zyklus einen falschen Überschuss sehen, kleine ASICs einschalten und sie dann abschalten müssen, wenn der grosse volle Leistung erreicht -> Netzbezug und Jojo. Die Variable prio_boot_ok verhindert das.

Kontinuierliches SHED - die Schlüsselkorrektur

Der Ausloser sustained_import feuert nur einmal. Um auf anhaltenden Bezug zu reagieren, enthält cycle_normal einen SHED-Zweig: wenn Netz > 50W und kleine ASICs übrig sind, schaltet er alle 5s einen ab (mit 30s Cooldown). Das bringt den Verbrauch schrittweise unter die Produktion, ohne Jojo.

```
# In cycle_normal:
- choose:
  - conditions:
    - "{{ grid_total > 50 and nb_small_on > 0
      and switch_cooldown_ok }}"
  sequence:
    - action: switch.turn_off
      target: {entity_id: "{{ small_on[-1] }}"}
```

07 Tagesbericht

Zuverlässiger Bericht auf der Home-Assistant-Seite

Jeden Tag um 20 Uhr sendet eine HA-Automation einen vollständigen Bericht über Telegram, ntfy und die HA-Companion-App. Anders als eine Browser-Version läuft sie 24/7: die Mining-Dauern sind daher immer exakt, auch wenn das Dashboard geschlossen ist.

history_stats-Sensoren

Home Assistant berechnet fortlaufend die EIN-Zeit jedes ASIC (Reset um Mitternacht). Diese Sensoren speisen den Bericht mit zuverlässigen Dauern.

```
sensor:
  - platform: history_stats
    name: "asic_big_1_heures_jour"
    entity_id: switch.asic_big_1
    state: "on"
    type: time
    start: "{{ today_at('00:00') }}"
    end: "{{ now() }}"
```

Berichtsinhalt

Block	Detail
Energie	Solar produziert, ASIC-Last, Netzbezug/-einspeisung (kWh)
Dauern	Gesamt-Mining + Stunden pro ASIC, nach Modell gruppiert (history_stats)
Erträge	Mining-Erträge des Vortages (T-1) in Euro
Netto	Erträge - Netzkosten (Einspeisung ohne Rückkaufvertrag nicht gezahlt)

Netto ohne Einspeisung: ohne EDF-Rückkaufvertrag wird eingespeiste Energie nicht vergütet. Netto = Mining-Erträge - Netzkosten. Die Einspeisung wird nur zur Information angezeigt.

Benachrichtigungskanäle

- **Telegram:** über einen Bot (Token + chat_id), Sofortnachricht
- **ntfy:** Open-Source-Push, eigenes Topic (ntfy.sh-App)
- **HA Companion:** native Benachrichtigung in der Home-Assistant-Mobil-App

08 HTML-Dashboard

Visuelle Echtzeit-Oberfläche

Eine einzige HTML-Datei (HTML + CSS + JS) für /config/www/. Sie liest die HA-API und zeigt Produktion, Verbrauch, animierten Energiefluss, Mining-Erträge und Verlauf.

#	Schritt	Detail
1	Datei kopieren	/config/www/dashboard_mining.html
2	secrets.example.js kopieren	-> secrets.js (Ihre Token und IDs)
3	API-Token erstellen	HA-Profil -> Sicherheit -> Langzeit-Token
4	Dashboard öffnen	http://HA_IP:8123/local/dashboard_mining.html

Anzupassende Variablen (secrets.js)

Variable	Standard	Ersetzen durch
HA_URL_LOCAL	http://192.168.1.XX:8123	Lokale IP Ihres HA
HA_URL_VPN	http://100.X.X.X:8123	Tailscale-IP (optional)
HA_TOKEN	eyJhbGciOi...	Ihr HA-API-Token
MINING_COINS	[...]	Liste Ihrer geminten Kryptos
F2POOL_TOKEN	ihr_token	Pool-API-Schlüssel

Dashboard-Funktionen

- **Animierter Energiefluss:** Solar -> Farm -> ASIC, mit Netzbezug/-einspeisung
- **Tagesdiagramm:** Leistungskurven (W) über 24h
- **Wochen-/Monatsdiagramm:** tagliche kWh-Balken (Solarman-Stil)
- **Mining-Erträge:** pro Krypto, in Echtzeit und in Euro
- **Bericht gestern:** Erträge - Netzkosten des Vortages
- **Mehrsprachig:** Französisch, Englisch, Deutsch, Spanisch
- **Nachster Start:** Countdown und Schwelle des nächsten ASIC

Balkendiagramme: für die Wochen- und Monatsansicht integriert das Dashboard die Leistung (W) in Energie (kWh) pro Tag mit der Trapezmethode und zeigt dann 4 Balken pro Tag (Solar, Farm, Bezug, Einspeisung) - genau wie die Solarman-App.

09 Mining-Pools

7 unterstützte Pools

Solar ASIC kann die Erträge jedes Pools anzeigen. Die Architektur ist universell: Home Assistant ruft die Daten serverseitig ab (kein CORS-Problem), das Dashboard liest die HA-Sensoren. Alle Sensoren müssen `sensor.f2pool_SYMBOL_day` und `sensor.f2pool_SYMBOL_unpaid` heißen (das Präfix `f2pool_` ist konventionell, unabhängig vom realen Pool).

Pool	Kryptos	Auth	Typ	Schwierigkeit
F2Pool	BTC, ALPH, KAS, LTC...	Token V2 (POST)	command_line	Schwer
K1Pool	KAS, RXD, ZIL...	Wallet (GET)	command_line	Einfach
2Miners	ETC, RVN, KAS, ZEC...	Wallet (GET)	rest:	Einfach
ViaBTC	BTC, ETH, LTC...	API Key (Header)	rest:	Mittel
NiceHash	BTC (alle Algos)	HMAC komplex	command_line	Schwer
HeroMiners	KAS, ALPH, RXD...	Wallet (GET)	rest:	Einfach
Kryptex	BTC, KAS, RXD...	Token (GET)	rest:	Mittel

Universelle Methode (jeder neue Pool)

1. API über PowerShell testen (Invoke-RestMethod) und die JSON-Struktur notieren
2. HA-Sensor erstellen: rest: bei einfachem GET, command_line: bei POST oder komplexer Auth
3. Template-Sensoren `f2pool_SYMBOL_day` und `f2pool_SYMBOL_unpaid` erstellen
4. Die Krypto zu `MINING_COINS` in `secrets.js` hinzufügen
5. HA neu starten und in Entwicklertools -> Zustände prüfen

2Miners-Beispiel (offentliches GET, Wallet)

Offentliche API ohne Authentifizierung: die Wallet-Adresse genügt. Subdomain pro Krypto (etc.2miners.com, kas.2miners.com...). Liefert Balance, 24h-Belohnung und Worker.

```
rest:
- resource: "https://etc.2miners.com/api/accounts/YOUR_WALLET"
  scan_interval: 300
  sensor:
    - name: "2miners_etc_raw"
      value_template: >
        {{ (value_json.stats.balance|float(0) / 1e9) | round(6) }}
      json_attributes: [stats, "24hreward", workersOnline]

template:
- sensor:
    - name: "f2pool_etc_day"      # generic dashboard name
      state: >
        {% set r = state_attr('sensor.2miners_etc_raw', '24hreward') %}
        {{ (r|float(0) / 1e9) | round(6) if r else 0 }}
```

Teiler je Coin: ETC = / 1.000.000.000, KAS = / 100.000.000, ZEC = / 100.000.000. Prüfen Sie immer die Rohantwort, bevor Sie den Teiler wählen.

ViaBTC-Beispiel (GET + API-Key-Header)

API-Schlüssel im Header (keine HMAC-Signatur). Getrennte Endpunkte für Hashrate und Gewinne.

```
rest:
- resource: "https://www.viabtc.com/res/openapi/v1/profit?coin=BTC"
  scan_interval: 300
  headers:
    Authorization: "YOUR_API_KEY_VIABTC"
  sensor:
    - name: "viabtc_btc_profit"
      value_template: "{{ value_json.data.total_profit|float(0) }}"
```

NiceHash-Beispiel (HMAC - Python-Skript)

NiceHash nutzt komplexe HMAC-Authentifizierung: jede Anfrage wird mit Timestamp + Nonce signiert. Direkt mit HA-Sensoren nicht möglich -> ein Python-Skript auf dem Pi, aufgerufen via `command_line`.

```
# /config/scripts/nicehash_balance.py
import hashlib, hmac, uuid, time, requests
API_KEY='...'; API_SECRET='...'; ORG_ID='...'
def sign(method, path, ts, nonce, body=''):
    msg = f'{API_KEY}\x00{ts}\x00{nonce}\x00\x00{ORG_ID}'
    msg += f'\x00\x00{method}\x00{path}\x00{body}'
    return hmac.new(API_SECRET.encode(), msg.encode(),
                    hashlib.sha256).hexdigest()
ts=str(int(time.time()*1000)); nonce=str(uuid.uuid4())
path='/main/api/v2/accounting/account2/BTC'
sig=sign('GET', path, ts, nonce)
headers={'X-Time':ts, 'X-Nonce':nonce, 'X-Organization-Id':ORG_ID,
        'X-Auth':f'{API_KEY}:{sig}'}
r=requests.get(f'https://api2.nicehash.com{path}', headers=headers)
print(r.json().get('totalBalance',{}).get('available','0'))
```

```
# configuration.yaml
command_line:
  - sensor:
      name: "nicehash_btc_balance"
      scan_interval: 300
      command: "python3 /config/scripts/nicehash_balance.py"
      value_template: "{{ value | float(0) }}"
      command_timeout: 20
```

Endpunkte der anderen Pools

Pool	Endpunkt	Schlusselfelder
K1Pool	GET /api/miner/{coin}/{wallet}	miner.pendingBalance, miner.coinsPerDay
HeroMiners	GET /api/stats_address?address={w}	stats.balance, stats.paid_today
Kryptex	GET /api/v1/hashrates?token={t}	data.balance, data.todayIncome
F2Pool	POST /v2/assets/balance	balance, yesterday_income

10 Fernzugriff

Tailscale VPN

Tailscale erstellt ein privates Netzwerk zwischen Ihren Geräten. Sie greifen von überall auf Ihr Dashboard zu (4G, externes WiFi) als wären Sie zu Hause, ohne einen Port zu öffnen.

#	Schritt	Detail
1	HA-Add-on	Einstellungen -> Add-ons -> Tailscale -> Starten
2	Tailscale-Konto	tailscale.com (gratis) -> die 100.x.x.x-IP notieren
3	Mobile App	Tailscale iOS/Android installieren, gleiches Konto
4	secrets.js bearbeiten	HA_URL_VPN mit Ihrer 100.x.x.x-IP setzen

```
const HA_URL_LOCAL = 'http://192.168.1.XX:8123';
const HA_URL_VPN   = 'http://100.X.X.X:8123';
// Auto-switch based on detected network
let HA_URL = location.hostname.startsWith('192.168')
  ? HA_URL_LOCAL : HA_URL_VPN;
```

Auto-Umschaltung: das Dashboard erkennt, ob es lokal (192.168.x.x) oder per Fernzugriff geöffnet wird, und wählt die richtige URL. Transparent für den Nutzer.

11 Fehlerbehebung

FAQ und Problemlosung

Problem	Lösung
Dashboard zeigt FEHLER	Ungultiges Token oder falsche IP. Token in HA neu erstellen und URL in secrets.js prüfen.
Sensoren unavailable	Templates neu laden: Entwicklertools -> YAML -> Template-Entitäten.
ASICs schalten nicht ein	Prüfen, ob die V37-Automation aktiv ist und der Überschuss die Schwelle für die nötige Dauer überschreitet.
Steckdosen-IDs geändert	Entwicklertools -> Zustände -> Filter 'switch.' für die neuen entity_id.
Tagesdiagramm leer	Normal in den ersten 24h. Der Verlauf sammelt sich schrittweise an.
Wochen-/Monatsdiagramm leer	recorder: purge_keep_days >= 31 prüfen und dass die Sensoren in include: sind.
Netzbezug trotz Überschuss	Der Filter ignoriert Bezug < 20W (Refoss-CT-Rauschen). Darüber die Zangen-Kalibrierung prüfen.
Krypto-Preise unknown	10 min warten (1. CoinGecko-Aufruf) und den Internetzugang des Pi prüfen.
20-Uhr-Bericht nicht erhalten	Telegram/ntfy-Zugangsdaten prüfen und dass die Bericht-Automation aktiv ist. Manuell testen.
ASIC-Jojo	Cooldown (30s) oder Prioritäts-Boot-Verzögerung (10 min) in der Automation erhöhen.
Über 4G nicht erreichbar	Tailscale muss auf dem Telefon UND auf HA aktiv sein. Die 100.x.x.x-IP in der App prüfen.

12 Gute Installation!

Dieses Projekt wird kostenlos geteilt. Verbessern Sie es, verteilen Sie es, teilen Sie es mit der Community.

Abschluss-Checkliste

- [V] Refoss EM06P in HA integriert (6 Leistungssensoren sichtbar)
- [V] configuration.yaml mit Templates, prioritären ASICs und Erträgen
- [V] V1.0.37-Automation aktiv (Multi-Priorität, SHED, Anti-Jojo)
- [V] Tagliche 20-Uhr-Bericht-Automation konfiguriert (Telegram/ntfy/HA)
- [V] HTML-Dashboard angepasst in /config/www/
- [V] secrets.js ausgefüllt (nie auf GitHub committen)
- [V] Tailscale-Fernzugriff (optional)

Solar ASIC v4 - Halo44 - Freie Lizenz.

Lasst uns sauber minen. Quellcode und Beispiele auf GitHub verfügbar.